Payroll Module

Detailed Swagger Testing Workflow

A controller-by-controller execution guide for setting up salary data, generating payroll, moving it through approval and payment, and validating each response.

What this covers

This guide is based on the controllers and services under **com.my_hourly.payroll**. It covers the order required for a successful happy-path test, alternate draft and regeneration flows, authorization, request examples, validation rules, and the expected state transitions.

Quick prerequisite checklist

Requirement	Why it matters	How to verify
JWT with HR_ADMIN or SUPER_ADMIN	All setup endpoints require an HR/Super Admin role. Payroll generation also allows PAYROLL_ADMIN.	POST /api/v1/auth/login; add accessToken in Swagger Authorize.
Active employee	Payment details and payroll generation reject inactive employees.	GET /api/v1/employees?page=0&size;=10
Employee joined by payroll month	A person who joined after the requested month cannot be processed.	Check dateOfJoining in the employee response.
Active salary template	Salary structure creation only accepts an active template.	POST template; then GET /salary-templates/{id}.
Active salary structure + payment details	Both are mandatory during payroll generation.	Use the payroll setup endpoints in Steps 4 and 5.

Recommended test identity

The local application properties configure an HR user: harshitha / Hr@1234. Use it only in a local development environment. The login response carries the JWT access token needed by Swagger.

1. Authenticate and choose an employee

Start outside the Payroll tags. The payroll endpoints are protected by Spring Security and Swagger is configured for Bearer JWT authentication.

1A. Login

Method	Endpoint	Role
POST	/api/v1/auth/login	Public

```
{
"usernameOrEmail": "harshitha",
"password": "Hr@1234"
}
```

Swagger action

Copy **data.accessToken** from the login response. Click **Authorize** in Swagger and paste the token. Swagger uses HTTP Bearer authentication, so it will send the Authorization header for the protected calls.

1B. Get an employee ID

Method	Endpoint	Allowed roles
GET	/api/v1/employees?page=0&size;=10	HR_ADMIN, MANAGER

Record a returned employee **id** (called **employeeId** below). Before continuing, confirm that the employee is active. The payroll service also rejects an employee whose **dateOfJoining** is later than the payroll month.

Controller map

Tag / Controller	Base path	Purpose in the workflow
17 - Employee Payment Details	/api/v1/payroll/payment-details	Bank and statutory payment snapshot source.
18 - Salary Template	/api/v1/payroll/salary-templates	Reusable earning/deduction master data.
19 - Salary Structure	/api/v1/payroll/salary-structures	Employee-specific assignment of a template.
20 - Payroll	/api/v1/payroll	Monthly payroll, lifecycle transitions, reads and PDF payslip.

2. Create master payroll data

Create the salary template first. Its values are copied into a salary structure and later into the payroll transaction, so later template changes do not rewrite historic payroll records.

2A. Create salary template

Method	Endpoint	Allowed roles
POST	/api/v1/payroll/salary-templates	HR_ADMIN, SUPER_ADMIN

```
{
"employeeType": "FULL_TIME",
"basicSalary": 30000, "hra": 12000, "specialAllowance": 5000,
"medicalAllowance": 1500, "travelAllowance": 2000, "bonus": 3000,
"otherAllowance": 1000, "pf": 1800, "esi": 0,
"professionalTax": 200, "incomeTax": 0, "otherDeduction": 0
}
```

Save response **id** as **salaryTemplateId**. A template is unique per employeeType; creating another template for the same type can fail as a duplicate. The template begins active.

2B. Create employee payment details

Method	Endpoint	Allowed roles
POST	/api/v1/payroll/payment-details	HR_ADMIN, SUPER_ADMIN

```
{
"employeeId": 1,
"panNumber": "ABCDE1234F",
"bankName": "State Bank of India",
"accountNumber": "123456789012",
"ifscCode": "SBIN0001234",
"paymentMode": "BANK_TRANSFER",
"uanNumber": "100123456789",
"pfNumber": "KNBNG1234567000001",
"esiNumber": "31001234567890001"
}
```

Payment-mode values: BANK_TRANSFER, UPI, CHEQUE, or CASH. Each employee can have only one payment-details record. Use PUT /api/v1/payroll/payment-details/{employeeId} to change it.

3. Assign salary and generate payroll

The salary structure converts the selected active template into an active, employee-owned salary setup. Payroll generation uses this record plus payment details.

3A. Create initial salary structure

Method	Endpoint	Allowed roles
POST	/api/v1/payroll/salary-structures	SUPER_ADMIN, HR_ADMIN, PAYROLL_ADMIN

```
{
"employeeId": 1,
"salaryTemplateId": 1,
"effectiveFrom": "2026-08-01",
"remarks": "Initial salary setup"
}
```

The employee may have only one active salary structure. Confirm it with **GET /api/v1/payroll/salary-structures/employee/{employeeId}/active**.

3B. Generate the monthly payroll

Method	Endpoint	Allowed roles
POST	/api/v1/payroll/generate	SUPER_ADMIN, HR_ADMIN, PAYROLL_ADMIN

```
{
"payrollMonth": "2026-08-01",
"employeeIds": [1],
"remarks": "August 2026 payroll",
"saveAsDraft": false
}
```

Use the first day of the target month. Omit or pass an empty employeeIds list to attempt generation for every active employee. The response is a summary: save a value from generatedPayrolls as payrollNumber; any failed employee appears under failedEmployees with a reason.

3C. Retrieve the generated record and its ID

Method	Endpoint	Use
GET	/api/v1/payroll/number/{payrollNumber}	Retrieve a response containing the payroll id.
GET	/api/v1/payroll/{id}	Recheck one payroll record at any point.
GET	/api/v1/payroll/month?payrollMonth=2026-08-01	Inspect all payrolls for a month.

4. Complete the normal approval and payment flow

When generated with saveAsDraft=false, the initial status is GENERATED. The normal lifecycle requires approval before payment.

Lifecycle

Current status	Allowed action	Result
GENERATED	PATCH /api/v1/payroll/{id}/approve	APPROVED; approvedDate becomes today.
APPROVED	PATCH /api/v1/payroll/{id}/pay?paymentReference=UTR-202608-001	PAID; paymentDate becomes today and reference is saved.
APPROVED or PAID	GET /api/v1/payroll/{id}/payslip	Downloads application/pdf.

4A. Approve

```
PATCH /api/v1/payroll/{id}/approve
No request body
```

4B. Download/verify payslip

```
GET /api/v1/payroll/{id}/payslip
```

4C. Mark paid

```
PATCH /api/v1/payroll/{id}/pay?paymentReference=UTR-202608-001
No request body
```

Important implementation detail

Although the source contains a MarkPayrollPaidRequest DTO, the current controller does **not** use it. Swagger correctly exposes only the required **paymentReference** query parameter. Do not submit paymentReference in a request body.

Useful reads

Endpoint	What it checks
GET /api/v1/payroll/employee/{employeeId}	Employee payroll history, including all versions and months.
GET /api/v1/payroll/status?status=PAID	Payrolls in a specific status.
GET /api/v1/payroll/month?payrollMonth=2026-08-01	All payrolls generated for a month.

5. Draft, revision, cancellation and regeneration tests

Draft test

Generate with **saveAsDraft: true**. The result starts in DRAFT and cannot be approved directly. Update it first if needed.

```
PUT /api/v1/payroll/{id}/draft
{
"totalWorkingDays": 22,
"workedDays": 20,
"lopDays": 2,
"remarks": "Two LOP days verified"
}
```

The update recalculates gross salary, LOP, total deduction and net payable. Constraint: workedDays + lopDays must not exceed totalWorkingDays. The controller exposes no transition from DRAFT to GENERATED, so a DRAFT cannot reach approval with the current API. Use a separate generated payroll for the full approve/pay test.

Salary revision test

Method	Endpoint	Rule
POST	/api/v1/payroll/salary-structures/revision	effectiveFrom must be after the active structure's effectiveFrom.

```
{
"employeeId": 1,
"salaryTemplateId": 2,
"effectiveFrom": "2026-09-01",
"remarks": "Annual revision"
}
```

This marks the prior structure INACTIVE and sets its effectiveTo to one day before the new effectiveFrom.

Cancellation and regeneration

Endpoint	Allowed	Effect
PATCH /api/v1/payroll/{id}/cancel	DRAFT or GENERATED only	Status CANCELLED; active=false.
POST /api/v1/payroll/{id}/regenerate	Active record except PAID	Old version becomes SUPERSEDED; a new GENERATED version is created.

Regeneration retains employee, bank, attendance and salary snapshots from the original payroll. It does not rebuild values from newly changed master data.

6. Validation, calculations and troubleshooting

Calculation used when generating

Value	Formula / default
Gross salary	Basic + HRA + Special + Medical + Travel + Bonus + Other allowance
Default attendance	22 total working days, 22 worked days, 0 LOP days
LOP deduction	(Gross / Total Working Days) x LOP Days; rounded HALF_UP to 2 decimals
Total deduction	PF + ESI + professional tax + income tax + other deduction + LOP
Net payable	Gross - total deduction

Common response failures

Symptom	Likely cause / resolution
401 or 403	Authenticate, use Swagger Authorize, and confirm the JWT has a permitted role.
Active Salary Structure not found	Create the structure using the employeeId and an active salaryTemplateId.
Payment details not found	Create employee payment details before calling /generate.
Active payroll already exists	Use another payrollMonth, cancel/regenerate as appropriate, or test with a different employee.
Only GENERATED payroll can be approved	The record is DRAFT or another status. Generate with saveAsDraft=false for approval testing.
Total deduction cannot exceed gross	Reduce deduction values in the template or draft update.
Invalid PAN	Use uppercase pattern ABCDE1234F, or omit/leave blank where accepted.

Final happy-path checklist

Login -> select active employee -> create salary template -> create payment details -> create salary structure -> generate payroll -> retrieve payroll ID -> approve -> download payslip -> mark paid -> GET by ID and confirm PAID.